Task 7 — Pay-per-Application Flow

AI / ML Engineer · WEEK 3 · PHASE 2 · Study Guide & Build Pipeline

1 · Where this fits

This task sits in **WEEK 3 · PHASE 2**. Across the whole team this day is about **Pay-per-Application Flow**. You own the intelligence layer of PlaceMux — the matching, ranking, parsing and recommendation models that turn verified skill scores into trustworthy job matches. Your job is not just to make a model that runs, but one whose decisions you can explain, measure and defend with numbers.

Your focus this task: Tune matching to protect paid-apply conversion.

By end of task, the founder should be able to verify: A student can pay ■100 and apply, end-to-end in test.

2 · Learning objectives

By the end of this study guide and task you will be able to:

- Deliver “Matching tune” to a demoable, real-data standard.
- Explain how your work is verified and how it scores out of 100.
- Avoid the specific failure modes that make this kind of work look 'done' when it isn't.

3 · Before you start (prerequisites & inputs)

Make sure you have these in place — missing inputs are the most common reason a task stalls:

- Python 3 and a working data-science environment (numpy, pandas, scikit-learn).
- Comfort with train/validation/test splits and the idea of a baseline.
- Basic understanding of precision, recall, false-positive rate and why a single accuracy number lies.
- Ability to read an API contract and turn it into a feature/inference call.
- Access to the agreed sample dataset (real-shaped, even if small) for this task.

Upstream dependency — confirm this is ready before you begin

- Waiting on: Baseline. If it's late, your task slips — chase it early and agree an ETA rather than waiting silently.

4 · Key concepts to understand first

Feature space

A feature is a measurable signal (a verified skill score, a JD requirement, years of exposure). The feature space is the full set of signals the model sees. Garbage or leaky features beat any fancy model — design these deliberately before modelling.

Baseline first

Before any clever model, build a dumb baseline (e.g. rank by overlap of required vs verified skills). Every later number is only meaningful relative to this baseline.

Explainability

For every match or score, you must be able to give a plain-English 'why'. A black box that says 'trust me' is a red flag in a hiring product where decisions affect people's careers.

Real metrics, not vibes

Report precision, recall, false-positive rate on real sample data — not 'it works'. A guardrail metric (e.g. relevance after a pricing change) protects the product from silent degradation.

Generalisation

Working on a toy example proves nothing. The question is whether it holds on unseen, real-shaped data and at the scale the marketplace will actually hit.

5 · Recommended stack & tools

- Python · pandas / numpy for data wrangling
- scikit-learn for classical models & metrics
- FastAPI (or the agreed serving layer) to expose inference
- A vector store / similarity search for discovery
- MLflow or a simple experiment log for runs & metrics
- Jupyter for exploration, then promote code into the repo

6 · The build pipeline (follow in order)

This is the flow to take the task from nothing to demoable. Work top to bottom; don't skip the test and verify stages — that's where 'looks done' becomes 'is done'.

Stage A — Understand & set up

1. Re-read what good looks like and the definition of done (Section 7) so you build to the target, not past it.
2. Confirm every prerequisite and upstream input above is actually in hand.
3. Pull the agreed sample data / contract and set up a clean branch and working environment.

Stage B — Build: Matching tune

1. Design the approach & baseline

Restate what good looks like for “Matching tune” and write down the baseline you'll beat and the metric you'll judge it by (precision / recall / false-positive rate).

2. Implement & train

Build “Matching tune” incrementally on the agreed sample data; keep the experiment log so every run's numbers are reproducible.

3. Measure on real sample data

Evaluate on held-out real-shaped data — never the data you tuned on — and record the numbers, not 'it works'.

4. Make it explainable & demoable

Produce a one-example walkthrough: this input, this output, and the plain-English reason — that's your demo.

Stage C — Integrate, verify & make demoable

1. Wire your deliverable(s) into the end-to-end flow and run the whole journey once, start to finish.
2. Ask them to walk you through one real example end-to-end: this student, this job, and why it's a match.
3. Aim for what 'good' looks like: They show numbers — match scores, accuracy, false-positive rates — on real sample data, not just ‘it works’.
4. Prepare a 2-minute live demo on real (even if small) data — not slides, not 'it works'.

7 · Definition of Done

You are done when all of these are true and demoable:

- Ranking tuned for conversion.

- “Matching tune” is complete, persisted/real, and demoable end-to-end.

8 · How your work is evaluated (scored out of 100)

How it's checked: Ask them to walk you through one real example end-to-end: this student, this job, and why it's a match.

Good looks like: They show numbers — match scores, accuracy, false-positive rates — on real sample data, not just ‘it works’.

Scoring parameter	Marks
Core deliverable — “Matching tune” built, working & demoable	50
Real-data quality & correctness (real sample data at scale, not a toy/happy-path)	20
Live verification & evidence (demoed live; real numbers, not claims)	15
Dependency, failure & edge-case handling (errors handled; hand-offs honoured)	15
TOTAL	100

9 · Pitfalls to avoid

Worry if any of these are true of your work

- The model is a black box, just trust it” — you should always get a plain-English ‘why’.
- Quality is described with no numbers and no baseline to compare against.
- It only works on a toy example, never on real sample data.
- We'll add payment failure handling later” — with real money, failure handling IS the feature, not an extra.
- Tuning to the demo dataset until it looks perfect, then watching it collapse on real data.
- Quoting one accuracy number with no baseline and no breakdown by segment.
- Shipping a model you cannot explain — in hiring, unexplained = untrusted = unusable.

10 · Hand-off & dependencies

- You hand off: Tuned matching. Make sure the next team can pick it up without you.
- You depended on: Baseline.

11 · Self-check before you submit

If you can answer 'yes, and I can show it live' to each, you're ready:

- Can you show me ‘Matching tune’ working live, rather than just describing it?
- Walk me through what happens if a payment fails halfway — does the student lose money OR the application?
- How do we know our records match exactly what the gateway says we collected?
- Are we in real-money mode or still test mode — and what's left before real money?

12 · Go deeper (optional further study)

- Precision/recall trade-offs and the PR curve
- Learning-to-rank basics (pointwise vs pairwise)
- Embeddings & approximate nearest-neighbour search
- Bias/fairness auditing for selection systems
- Model drift detection and retraining triggers